

Absolutely. Since SQL is now part of your Day 5 backend/database progression, I want you to learn it from first principles rather than memorizing queries.

The notes below are written as novice → practical developer → interview-ready. They focus on the SQL concepts you will actually need when moving your Project 1 from in-memory storage to PostgreSQL. Your Day 5 material specifically identifies relational databases, tables, keys, constraints, CRUD SQL, transactions, and indexes as the required database foundation.

SQL — Detailed Beginner-to-Interview Notes

1. First: What Problem Does SQL Solve?

Imagine you are building your AI Engineering Change & Test Intelligence Platform.

You need to store documents:

```
Document 1
ID: doc-001
Name: Engine Requirement
Type: requirement
Version: 1.0
Status: active
```

```
Document 2
ID: doc-002
Name: Engine Architecture
Type: design
Version: 1.0
Status: active
```

At first, Python can store this in memory:

```
documents = {
    "doc-001": {...},
    "doc-002": {...}
}
```

But there is a problem.

When your application stops:

```
Python process
  ↓
shutdown
  ↓
memory disappears
  ↓
documents disappear
```

You need persistent storage.

That's where a database comes in.

2. What Is a Database?

A database is a system for storing and retrieving data in a structured and reliable way.

Think of it as a sophisticated storage system.

Instead of:

Python Dictionary

you have:

```
Application
  ↓
Database
  ↓
Persistent storage
```

A database gives you capabilities such as:

- persistent storage
- querying
- updating
- deleting
- concurrency
- transactions
- constraints
- indexing
- security
- recovery

3. What Is a Relational Database?

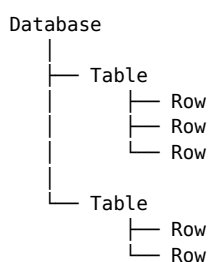
A relational database stores data primarily in tables and allows relationships between tables.

Examples include:

- PostgreSQL
- MySQL
- SQLite
- Oracle Database
- Microsoft SQL Server

For your project, you're using PostgreSQL.

The basic mental model is:



4. What Is SQL?

SQL stands for:

```
**Structured Query Language**
```

SQL is the language we use to communicate with relational databases.

For example:

```
SELECT * FROM documents;
```

means:

```
"Give me all records from the `documents` table."
```

Another example:

```
DELETE FROM documents  
WHERE id = 'doc-001';
```

means:

```
"Delete the document whose ID is `doc-001`."
```

So think:

```
Python application  
  ↓  
  SQL  
  ↓  
PostgreSQL
```

5. SQL vs PostgreSQL

This distinction is important.

SQL

SQL is a language.

PostgreSQL

PostgreSQL is a database management system that understands SQL.

Analogy:

```
English = language  
PostgreSQL = system that understands the language
```

You write:

```
SELECT * FROM documents;
```

PostgreSQL executes it.

6. Database → Schema → Table

There are several layers.

Think of a company filing system.

```
Database
↓
Schema
↓
Table
↓
Row
↓
Column
```

For example:

```
project_database
↓
public schema
↓
documents
```

Your documents table could contain:

```
| id | name | type | version | status |
|---|---|---|---|---|
| doc-001 | Engine Requirement | requirement | 1.0 | active |
| doc-002 | Engine Design | design | 1.0 | active |
| doc-003 | Engine Test | test | 1.0 | draft |
```

7. What Is a Table?

A table is a structured collection of related records.

For example:

```
documents
```

is a table.

It represents one logical entity:

Documents stored by our application.

Another table could be:

```
users
```

representing users.

Another:

```
test_cases
```

representing test cases.

8. What Is a Row?

A row represents one record.

Example:

```
| id | name | type |  
|---|---|---|  
| doc-001 | Engine Requirement | requirement |
```

This entire row represents one document.

Think:

Row = one instance of an entity

9. What Is a Column?

A column represents one property/attribute.

For example:

```
id  
name  
document_type  
version  
status
```

Each column has a meaning.

```
id           → identity  
name         → document name  
document_type → requirement/design/test  
version      → document version  
status       → current state
```

10. A Complete Example

Let's create a simple table.

```
CREATE TABLE documents (  
  id VARCHAR(100),  
  name VARCHAR(255),  
  document_type VARCHAR(50),  
  version VARCHAR(20),  
  status VARCHAR(50)  
);
```

We now have:

```
documents  
|  
|— id  
|— name  
|— document_type  
|— version  
|— status
```

11. Data Types

Every column needs a data type.

Common PostgreSQL types include:

INTEGER

```
age INTEGER
```

Stores whole numbers.

```
10
25
100
```

BIGINT

For larger integers.

```
id BIGINT
```

DECIMAL

For exact decimal values.

```
price DECIMAL(10,2)
```

BOOLEAN

```
is_active BOOLEAN
```

Values:

```
TRUE
FALSE
```

TEXT

```
description TEXT
```

Variable-length text.

VARCHAR

```
name VARCHAR(255)
```

String with a maximum length.

DATE

```
created_date DATE
```

Example:

```
2026-09-14
```

TIMESTAMP

`created_at` `TIMESTAMP`

Stores date + time.

12. Why Data Types Matter

Suppose you store:

`age`

as text.

You could accidentally store:

`"twenty"`

But if it's:

`age` `INTEGER`

the database knows:

This must be a number.

This is one part of data integrity.

13. NULL

NULL means:

The value is missing/unknown/not provided.

It does not mean:

`0`

and it does not mean:

`empty string`

For example:

<code>id</code>	<code>name</code>	<code>version</code>
<code>doc-001</code>	<code>Engine Requirement</code>	<code>NULL</code>

The version isn't necessarily "NULL".

It means:

No value is currently stored.

14. NOT NULL

If a field must always exist:

```
name TEXT NOT NULL
```

Now:

```
INSERT INTO documents (id)
VALUES ('doc-001');
```

would fail because:

```
name
```

is required.

The database is enforcing a rule.

15. PRIMARY KEY

This is one of the most important SQL concepts.

A primary key uniquely identifies each row.

Example:

```
id VARCHAR(100) PRIMARY KEY
```

Now:

```
doc-001
doc-002
doc-003
```

can uniquely identify documents.

The primary key provides uniqueness and identity and is particularly useful for lookups and relationships.

16. Why Do We Need a Primary Key?

Imagine:

name	type
Engine Requirement	requirement
Engine Requirement	requirement

Which one is:

```
GET /documents/???
```

You need a unique identifier.

Therefore:

```
id
```

is usually used.

17. Primary Key Rules

A primary key:

- must be unique
- cannot be NULL
- identifies a row
- is commonly indexed by the database

Example:

```
CREATE TABLE documents (  
  id TEXT PRIMARY KEY,  
  name TEXT NOT NULL  
);
```

18. Why Not Use Name as Primary Key?

You might think:

```
name TEXT PRIMARY KEY
```

But:

```
Engine Requirement  
Engine Requirement
```

could potentially exist more than once.

Also, names can change.

An identifier should represent identity, not necessarily human-readable naming.

Your Day 5 material explicitly calls out why the document name should not simply be used as the primary key.

19. UNIQUE

UNIQUE means:

This column cannot contain duplicate values.

Example:

```
email TEXT UNIQUE
```

You can have:

```
alice@gmail.com  
bob@gmail.com
```

but not:

alice@gmail.com
alice@gmail.com

20. PRIMARY KEY vs UNIQUE

Both enforce uniqueness.

But conceptually:

PRIMARY KEY
↓
Main identity of row

while:

UNIQUE
↓
Another value that must also be unique

Example:

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  email TEXT UNIQUE  
);
```

Here:

id → identity
email → unique business attribute

21. CHECK Constraint

A CHECK constraint enforces a condition.

Suppose your document types are:

requirement
design
test

You could write:

```
CHECK (  
  document_type IN ('requirement', 'design', 'test')  
)
```

Now:

requirement ☐
design ☐
test ☐
banana ☐

Your Day 5 material specifically recommends this type of constraint for document_type.

22. FOREIGN KEY

Now we reach relationships.

Suppose you have:

```
documents
```

and:

```
test_cases
```

A test case might belong to a document.

```
documents
-----
id
name

test_cases
-----
id
document_id
name
```

We can define:

```
document_id TEXT REFERENCES documents(id)
```

This is a foreign key.

It says:

```
`test_cases.document_id` must refer to an existing document.
```

23. Why Foreign Keys Matter

Without a foreign key, you might accidentally create:

```
test_case
document_id = doc-999
```

when:

```
doc-999
```

doesn't exist.

The database can prevent that.

24. CRUD

One of the most important concepts in backend development is CRUD.

CRUD means:

```
C → Create
R → Read
U → Update
D → Delete
```

SQL maps naturally to CRUD:

```
| Operation | SQL |
|---|---|
| Create | `INSERT` |
| Read | `SELECT` |
| Update | `UPDATE` |
| Delete | `DELETE` |
```

Your Day 5 material specifically requires these four SQL operations.

25. INSERT — Create Data

Suppose:

```
CREATE TABLE documents (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  document_type TEXT NOT NULL,
  version TEXT,
  status TEXT
);
```

Insert:

```
INSERT INTO documents (
  id,
  name,
  document_type,
  version,
  status
)
VALUES (
  'doc-001',
  'Engine Requirement',
  'requirement',
  '1.0',
  'active'
);
```

Now PostgreSQL stores the row.

26. INSERT Another Record

```
INSERT INTO documents (
  id,
  name,
  document_type,
  version,
  status
)
VALUES (
  'doc-002',
  'Engine Architecture',
  'design',
  '1.0',
  'active'
);
```

Now:

```
| id | name | type |
|---|---|---|
| doc-001 | Engine Requirement | requirement |
| doc-002 | Engine Architecture | design |
```

27. SELECT — Read Data

The most fundamental query:

```
SELECT *  
FROM documents;
```

Meaning:

```
SELECT  
  ↓  
which columns?  
  
*  
  ↓  
all columns  
  
FROM  
  ↓  
which table?  
  
documents
```

28. Selecting Specific Columns

Instead of:

```
SELECT *  
FROM documents;
```

you can write:

```
SELECT id, name, document_type  
FROM documents;
```

This returns only:

```
id  
name  
document_type
```

Generally, explicitly selecting the columns you need is preferable to blindly using `*` in application queries.

29. WHERE

WHERE filters rows.

Example:

```
SELECT *  
FROM documents  
WHERE id = 'doc-001';
```

Meaning:

Find the document whose ID is ``doc-001``.

30. WHERE with Text

```
SELECT *  
FROM documents  
WHERE document_type = 'requirement';
```

Result:

Only requirement documents

31. WHERE with Numbers

Suppose:

version_number

is numeric.

You could write:

```
SELECT *  
FROM documents  
WHERE version_number > 2;
```

32. AND

You can combine conditions:

```
SELECT *  
FROM documents  
WHERE document_type = 'requirement'  
AND status = 'active';
```

Both conditions must be true.

33. OR

```
SELECT *  
FROM documents  
WHERE document_type = 'requirement'  
OR document_type = 'design';
```

Either condition can be true.

34. NOT

```
SELECT *  
FROM documents
```

```
WHERE status != 'deleted';
```

This means:

Return documents whose status is not deleted.

35. ORDER BY

Suppose you want documents ordered by name:

```
SELECT *  
FROM documents  
ORDER BY name;
```

Descending:

```
SELECT *  
FROM documents  
ORDER BY name DESC;
```

Ascending:

```
SELECT *  
FROM documents  
ORDER BY name ASC;
```

36. LIMIT

Suppose you only want 10 rows:

```
SELECT *  
FROM documents  
LIMIT 10;
```

Very important when dealing with large datasets.

37. OFFSET

You can skip rows:

```
SELECT *  
FROM documents  
LIMIT 10  
OFFSET 20;
```

Conceptually:

```
skip 20  
take next 10
```

This is one basic approach to pagination.

38. UPDATE

Suppose:

```
doc-001
```

currently has:

```
version = 1.0
```

Update it:

```
UPDATE documents  
SET version = '2.0'  
WHERE id = 'doc-001';
```

The WHERE is extremely important.

39. The Dangerous UPDATE Mistake

Never casually write:

```
UPDATE documents  
SET status = 'active';
```

This updates every row.

You might have intended:

```
one document
```

but accidentally updated:

```
1,000,000 documents
```

Always understand which rows your query affects.

40. DELETE

Delete one document:

```
DELETE FROM documents  
WHERE id = 'doc-001';
```

Again, WHERE matters.

41. The Dangerous DELETE Mistake

This:

```
DELETE FROM documents;
```

means:

Delete every row in the table.

Not:

Delete some documents.

This is why destructive operations require caution.

42. SQL Command Categories

You will hear these terms in interviews.

DDL

Data Definition Language

Used to define database structures.

Examples:

```
CREATE
ALTER
DROP
```

DML

Data Manipulation Language

Used to modify data.

Examples:

```
INSERT
UPDATE
DELETE
```

DQL

Often used to describe querying:

```
SELECT
```

TCL

Transaction Control Language:

```
COMMIT
ROLLBACK
```

43. CREATE TABLE

Example:

```
CREATE TABLE documents (  
  id TEXT PRIMARY KEY,  
  name TEXT NOT NULL,  
  document_type TEXT NOT NULL,  
  version TEXT,  
  status TEXT  
);
```

This creates the structure.

44. ALTER TABLE

Suppose later you need:

```
description
```

You can add:

```
ALTER TABLE documents  
ADD COLUMN description TEXT;
```

45. DROP TABLE

```
DROP TABLE documents;
```

This removes the table.

Be extremely careful.

This is a destructive operation.

46. What Is a Transaction?

This is one of the most important backend/database concepts.

Suppose you perform two operations:

```
Operation 1  
Operation 2
```

You want both to succeed.

Instead of treating them independently:

```
Operation 1 → success  
Operation 2 → failure
```

you can group them into a transaction.

Conceptually:

```
BEGIN
↓
Operation 1
↓
Operation 2
↓
COMMIT
```

If something fails:

```
BEGIN
↓
Operation 1
↓
Operation 2 ❌
↓
ROLLBACK
```

Your Day 5 material defines a transaction as allowing multiple database operations to behave as one logical unit.

47. COMMIT

COMMIT means:

Make the transaction's changes permanent.

Conceptually:

```
BEGIN;

UPDATE ...;
INSERT ...;

COMMIT;
```

48. ROLLBACK

ROLLBACK means:

Undo the changes made in the current transaction.

Conceptually:

```
BEGIN;

UPDATE ...;

-- something fails

ROLLBACK;
```

The database returns to the transaction's previous state.

49. Why Transactions Matter

Imagine your AI platform processes a release change.

It needs to:

1. Create change record
2. Create impact-analysis record
3. Create test-generation record

Suppose:

- 1 → succeeds
- 2 → succeeds
- 3 → fails

You may not want:

change record	☐
impact record	☐
test record	☐

depending on your consistency requirements.

A transaction allows you to reason about these operations as one unit.

50. ACID

Once you understand transactions, learn ACID.

ACID stands for:

- A → Atomicity
- C → Consistency
- I → Isolation
- D → Durability

This is essential interview knowledge.

51. Atomicity

Atomicity means:

All operations in the transaction happen, or none of them do.

Think:

ALL
or
NOTHING

52. Consistency

Consistency means:

A transaction should move the database from one valid state to another valid state while respecting defined constraints/invariants.

For example:

```
foreign key
NOT NULL
UNIQUE
CHECK
```

should remain satisfied.

53. Isolation

Isolation concerns concurrent transactions.

Imagine:

```
User A
  ↓
updates document

User B
  ↓
reads document
```

What does B see while A's transaction is in progress?

That's part of transaction isolation.

PostgreSQL provides isolation mechanisms to manage concurrent operations.

54. Durability

Once committed:

```
COMMIT
```

the data should survive things such as a normal application restart.

That's one of the major differences between:

```
Python dictionary
```

and:

```
PostgreSQL
```

55. What Is an Index?

Imagine a table containing:

```
10 million documents
```

You frequently execute:

```
SELECT *
FROM documents
WHERE id = 'doc-987654';
```

Without an appropriate index, the database may need to inspect many rows.

An index provides a specialized data structure that can make certain lookups much faster.

56. Simple Index Example

```
CREATE INDEX idx_documents_type
ON documents(document_type);
```

Now PostgreSQL has an index associated with:

```
document_type
```

Your Day 5 material emphasizes the basic trade-off: indexes can speed reads but consume storage and can increase write/update cost.

57. Why Don't We Index Everything?

Because indexes aren't free.

Imagine:

```
Table
↓
10 columns
↓
10 indexes
```

Every time you insert/update data, indexes may also need to be maintained.

So:

```
More indexes
↓
Potentially faster reads
+
More storage
+
More write overhead
```

Database engineering is about trade-offs.

58. Primary Key and Indexes

When you define:

```
id TEXT PRIMARY KEY
```

the database creates the appropriate uniqueness enforcement/indexing structure for the primary key.

Therefore, you generally don't need to manually create another identical index on `id`.

59. Query Execution — Basic Mental Model

When you execute:

```
SELECT *  
FROM documents  
WHERE id = 'doc-001';
```

conceptually:

```
SQL query  
↓  
Parser  
↓  
Query planner/optimizer  
↓  
Execution plan  
↓  
Scan/index access  
↓  
Rows  
↓  
Result
```

You don't need to understand PostgreSQL's internals deeply yet.

But understand that SQL is declarative.

60. Declarative vs Imperative

This is an excellent interview concept.

Python:

```
for document in documents:  
    if document.id == "doc-001":  
        return document
```

You're telling the computer how to search.

SQL:

```
SELECT *  
FROM documents  
WHERE id = 'doc-001';
```

You're describing what data you want.

PostgreSQL decides how to execute the query.

61. SQL NULL Is Special

Consider:

```
SELECT *  
FROM documents  
WHERE version = NULL;
```

This does not work as beginners often expect.

Use:

```
SELECT *
```

```
FROM documents
WHERE version IS NULL;
```

And:

```
WHERE version IS NOT NULL;
```

This is an important interview/detail point.

62. Aggregate Functions

SQL can calculate things.

COUNT

```
SELECT COUNT(*)
FROM documents;
```

Means:

How many rows?

MAX

```
SELECT MAX(version_number)
FROM documents;
```

MIN

```
SELECT MIN(version_number)
FROM documents;
```

AVG

```
SELECT AVG(score)
FROM test_results;
```

SUM

```
SELECT SUM(cost)
FROM executions;
```

63. GROUP BY

Suppose you want to know:

How many documents exist of each type?

You can conceptually write:

```
SELECT document_type, COUNT(*)
FROM documents
GROUP BY document_type;
```

Result:

```
requirement    120
design          80
test           300
```

This becomes very useful in analytics.

64. HAVING

WHERE filters rows before grouping.

HAVING filters groups after grouping.

Example:

```
SELECT document_type, COUNT(*)
FROM documents
GROUP BY document_type
HAVING COUNT(*) > 100;
```

Meaning:

Show document types having more than 100 documents.

65. JOIN — The Most Important Advanced Beginner Concept

Relational databases become powerful when multiple tables are involved.

Suppose:

documents

```
| id | name |
|---|---|
| doc-001 | Engine Requirement |
| doc-002 | Engine Design |
```

test_cases

```
| id | document_id | name |
|---|---|---|
| test-001 | doc-001 | Engine Start Test |
| test-002 | doc-001 | Engine Stop Test |
```

We want:

```
document
+
its test cases
```

That's where JOIN comes in.

66. INNER JOIN

Conceptually:

```
SELECT
    documents.name,
    test_cases.name
FROM documents
INNER JOIN test_cases
    ON documents.id = test_cases.document_id;
```

The important idea:

```
documents.id
    =
test_cases.document_id
```

The database combines related rows.

67. Why JOINS Matter for Your Project

Eventually Project 1 may have tables like:

```
documents
requirements
code_changes
test_cases
test_runs
test_results
```

You will need relationships:

```
Requirement
  ↓
Code Change
  ↓
Impact Analysis
  ↓
Generated Test
  ↓
Test Run
  ↓
Test Result
```

SQL relationships and joins become fundamental.

68. One-to-Many Relationship

Example:

```
One document
  ↓
Many test cases
```

Database:

```
documents
-----
```

```
id
test_cases
-----
id
document_id
```

This is:

One-to-many.

69. One-to-One

Example:

```
User
↓
UserProfile
```

One user has one profile.

70. Many-to-Many

Suppose:

```
requirements
```

can map to:

```
test_cases
```

and one test case can validate multiple requirements.

That's:

```
Requirement ↔ Test Case
```

Relational databases typically model this using a junction/association table:

```
requirement_test_case
-----
requirement_id
test_case_id
```

This will become very relevant to your eventual Requirement → Code → Test → Result traceability system.

71. SQL Injection

This is critical for backend engineering.

Never build SQL like:

```
query = f"""
SELECT * FROM documents
WHERE id = '{document_id}'
"""
```

Suppose someone sends malicious input.

This can result in SQL injection.

72. Parameterized Queries

Instead, use parameterized queries through your database library.

Conceptually:

```
SELECT *  
FROM documents  
WHERE id = %s;
```

and provide the value separately.

The database driver handles parameter binding.

Your Day 5 notes already use %s in their example for this reason.

The principle is:

****Never construct SQL by blindly concatenating user-controlled values into the query string.****

73. SQL and FastAPI

Now connect the theory to your application.

Your architecture should be:

```
HTTP Request  
↓  
FastAPI Router  
↓  
Service  
↓  
Repository  
↓  
SQL  
↓  
PostgreSQL
```

For example:

```
POST /documents
```

becomes:

```
FastAPI  
↓  
DocumentService  
↓  
DocumentRepository  
↓  
INSERT  
↓  
PostgreSQL
```

This is exactly the persistence transition planned for your Day 5 project.

74. Why Should SQL Be in the Repository?

You don't want:

```
@app.post("/documents")
def create_document(...):
    # SQL here
```

and you don't want business logic mixed with SQL.

Instead:

```
Router
  ↓
Service
  ↓
Repository
```

The repository owns persistence concerns.

For example:

```
DocumentRepository.create()
DocumentRepository.get()
DocumentRepository.update()
DocumentRepository.delete()
```

The service doesn't need to know whether storage is:

PostgreSQL

or:

another database

or:

in-memory

That's abstraction.

75. The Most Important Architectural Idea

Your project is transitioning from:

```
DocumentRepository
  ↓
Python dictionary
```

to:

```
DocumentRepository
  ↓
PostgreSQL
```

The API should remain:

```
POST /documents
GET /documents
GET /documents/{id}
PUT /documents/{id}
DELETE /documents/{id}
```

The user of the API should not care about the persistence implementation.

That's a very good software-engineering lesson.

76. Connection Management

Your application needs a connection to PostgreSQL.

Conceptually:

```
FastAPI
  ↓
Database connection
  ↓
PostgreSQL
```

But you shouldn't casually create uncontrolled connections everywhere.

You need to understand:

- connection lifecycle
- opening connections
- using connections
- committing/rolling back
- closing connections
- connection pooling

Your Day 5 plan explicitly identifies connection management and the database connection layer as learning objectives.

77. Connection Pooling

Imagine:

100 requests

If every request creates a completely new database connection:

```
Request
  ↓
connect
  ↓
query
  ↓
disconnect
```

that's inefficient.

A connection pool maintains reusable connections.

Conceptually:

```
Application → Pool
               ├── connection 1
               ├── connection 2
               └── connection 3
```

└─ connection 4

Requests borrow connections and return them.

You don't need sophisticated pooling configuration on Day 5, but you should understand the concept.

78. Database Failure

What happens if PostgreSQL is unavailable?

```
FastAPI
  ↓
Repository
  ↓
PostgreSQL ☐
```

Your application needs to:

- handle the exception
- avoid leaking connections
- return an appropriate error
- log the failure
- expose useful observability

Later in your roadmap this will connect to:

```
OpenTelemetry
Prometheus
Grafana
```

and reliability engineering.

79. Database Testing

Your current project already has API tests, and Day 5 requires moving those tests toward PostgreSQL-backed persistence.

The fundamental question is:

How do we test database behavior reliably?

You need to think about:

```
Test database
Fixtures
Setup
Teardown
Transactions
Rollback
Isolation
```

80. Test Isolation

Imagine:

Test A

```
creates doc-001
```

Test B

expects:

```
database is empty
```

But if Test A's data remains:

```
Test B
  ↓
unexpected doc-001
```

Now your tests depend on execution order.

That's bad.

Each test should have a predictable environment.

81. SQL Injection + Security

For Google-level backend engineering, SQL knowledge is not only about syntax.

You should think about:

```
SQL Injection
  ↓
Parameterized queries
  ↓
Least privilege
  ↓
Secrets management
  ↓
Database access controls
```

Your Day 5 material also explicitly requires keeping database credentials outside Git and using environment-based configuration.

82. Environment Variables

Never put:

```
DATABASE_PASSWORD = "secret123"
```

in source code.

Instead:

```
DATABASE_URL
```

can be provided through the environment.

Conceptually:

```
Application
  ↓
Environment variable
  ↓
DATABASE_URL
  ↓
PostgreSQL
```

Your `.env` should not be committed.

Use:

```
.env.example
```

to document required configuration without exposing credentials.

83. SQL vs ORM

You will eventually encounter ORMs such as:

```
SQLAlchemy
```

An ORM allows you to work with database records using Python objects.

For example, instead of writing raw SQL:

```
SELECT *
FROM documents
WHERE id = ...;
```

you may use ORM APIs.

But learn SQL first.

Why?

Because if you don't understand SQL, an ORM can hide important concepts:

```
JOIN
INDEX
TRANSACTION
QUERY PLAN
CONSTRAINT
LOCK
```

For your Google preparation, SQL fundamentals are non-negotiable.

84. SQL Mental Model

When you see a SQL query, mentally break it down.

Example:

```
SELECT id, name
FROM documents
WHERE document_type = 'requirement'
ORDER BY name
LIMIT 10;
```

Read it as:

SELECT
↓
What do I want?

FROM
↓
Where is the data?

WHERE
↓
Which rows?

ORDER BY
↓
How should results be ordered?

LIMIT
↓
How many?

This mental model makes SQL much easier.

85. SQL Execution Order

An interview favorite.

Although we write:

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
LIMIT
```

the conceptual logical processing order is roughly:

```
FROM
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
ORDER BY
↓
LIMIT
```

You don't need to memorize this mechanically yet.

Understand why it matters.

86. WHERE vs HAVING

This is frequently asked.

WHERE

Filters individual rows.

```
WHERE status = 'active'
```

HAVING

Filters groups.

```
HAVING COUNT(*) > 10
```

Mental shortcut:

```
WHERE → rows  
HAVING → groups
```

87. DELETE vs DROP

Very important.

DELETE

```
DELETE FROM documents  
WHERE id = 'doc-001';
```

Deletes rows.

DROP

```
DROP TABLE documents;
```

Removes the table structure itself.

Think:

```
DELETE → remove data  
DROP → remove database object
```

88. DELETE vs TRUNCATE

Another useful distinction.

DELETE

```
DELETE FROM documents;
```

Deletes rows and can be filtered using WHERE.

TRUNCATE

```
TRUNCATE TABLE documents;
```

Removes all rows efficiently.

You should know the concept now; detailed PostgreSQL behavior can come later.

89. UPDATE and DELETE Safety

Always ask:

Which rows does this affect?

Before:

```
UPDATE documents
SET status = 'active'
WHERE ...
```

OR:

```
DELETE FROM documents
WHERE ...
```

you should understand the predicate.

For production engineering, accidental mass updates/deletes are serious failures.

90. The SQL You Absolutely Must Know

For your current level, become fluent with:

CREATE TABLE

INSERT INTO

SELECT
FROM
WHERE

UPDATE
SET
WHERE

DELETE FROM
WHERE

ORDER BY

GROUP BY

HAVING

LIMIT
OFFSET

JOIN

COUNT
SUM
AVG
MIN
MAX

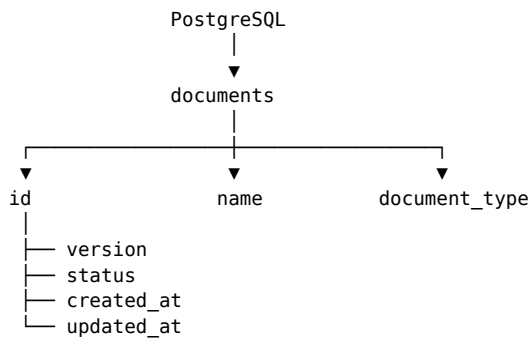
CREATE INDEX

BEGIN
COMMIT
ROLLBACK

You don't need advanced SQL yet.

91. Your Project's `documents` Table

For your current Project 1 stage, visualize the database as:



Your Day 5 plan explicitly proposes these document fields as the initial schema.

92. End-to-End Example

Imagine the client sends:

```
POST /documents
```

with:

```
{
  "id": "doc-001",
  "name": "Engine Requirement",
  "document_type": "requirement"
}
```

Your architecture:

```
Client
  ↓
FastAPI
  ↓
Pydantic validation
  ↓
DocumentService
  ↓
DocumentRepository
  ↓
SQL INSERT
  ↓
PostgreSQL
```

SQL might conceptually be:

```
INSERT INTO documents (
  id,
  name,
  document_type
)
VALUES (
  %s,
  %s,
  %s
);
```

Then PostgreSQL stores it.

93. GET Example

Client:

```
GET /documents/doc-001
```

Application:

```
FastAPI
  ↓
Service
  ↓
Repository
  ↓
SELECT
  ↓
PostgreSQL
```

SQL:

```
SELECT
    id,
    name,
    document_type,
    version,
    status
FROM documents
WHERE id = %s;
```

If no row exists:

```
Repository
  ↓
not found
  ↓
DocumentNotFoundError
  ↓
global exception handler
  ↓
404
```

This preserves the error architecture already established in your project.

94. The Most Important SQL Interview Questions for You

At this stage, you should be able to answer these without notes:

Beginner

1. What is SQL?
2. What is PostgreSQL?
3. What is a relational database?
4. What is a table?
5. What is a row?
6. What is a column?

7. What is a primary key?
8. What is a foreign key?
9. What is a constraint?
10. What is NULL?

CRUD

11. What is CRUD?
12. How do you insert a row?
13. How do you retrieve rows?
14. How do you update a row?
15. How do you delete a row?
16. Why is WHERE important?

Database design

17. Why use a primary key?
18. Primary key vs unique key?
19. What is NOT NULL?
20. What is a CHECK constraint?
21. What is a foreign key?
22. What is a one-to-many relationship?

Querying

23. What does JOIN do?
24. What is INNER JOIN?
25. What is GROUP BY?
26. What is HAVING?
27. WHERE vs HAVING?
28. What does ORDER BY do?
29. What does LIMIT do?

Performance

30. What is an index?
31. Why does an index make queries faster?
32. Why don't we index every column?
33. What is the trade-off of an index?

Transactions

34. What is a transaction?
35. What is ACID?
36. What is atomicity?

- 37. What is consistency?
- 38. What is isolation?
- 39. What is durability?
- 40. What are COMMIT and ROLLBACK?

Backend

- 41. Why should SQL live behind a repository?
- 42. How does FastAPI communicate with PostgreSQL?
- 43. What is connection pooling?
- 44. How do you prevent SQL injection?
- 45. How do you test database-backed APIs?
- 46. How do you isolate database tests?
- 47. What happens if PostgreSQL becomes unavailable?

95. Your SQL Learning Progression

Don't try to learn all of SQL in one day.

Your progression should be:

LEVEL 1 – Today

- Database fundamentals
- Tables
- Rows
- Columns
- Data types
- Primary keys
- Foreign keys
- Constraints



LEVEL 2

- INSERT
- SELECT
- WHERE
- UPDATE
- DELETE



LEVEL 3

- ORDER BY
- LIMIT
- GROUP BY
- HAVING
- Aggregate functions



LEVEL 4

- JOINS
- Relationships
- Subqueries
- CTEs



LEVEL 5

- Transactions
- ACID
- Isolation
- Locks

▼
LEVEL 6

- Indexes
- Query plans
- EXPLAIN
- Performance

▼
LEVEL 7

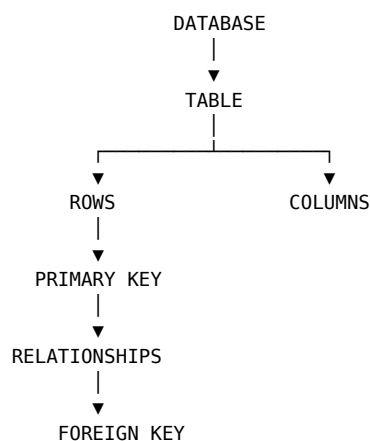
- PostgreSQL internals
- MVCC
- Vacuum
- Partitioning
- Replication
- High availability

Don't jump to Level 7 now.

Your immediate objective is to become very strong at Levels 1–3, understand Level 4, and have a solid conceptual understanding of Levels 5–6.

96. The One Mental Model I Want You to Remember

When you work with SQL, always think:



Then:

```

CREATE
↓
INSERT
↓
SELECT
↓
UPDATE
↓
DELETE
  
```

And when multiple operations must behave as one unit:

```

BEGIN
  
```

↓
operations
↓
COMMIT

or:

BEGIN
↓
failure
↓
ROLLBACK

And when reads become expensive:

Query
↓
Index
↓
Faster lookup

but:

Index
↓
extra storage + write maintenance

That is the foundation.

97. Your Day 5 SQL "Definition of Understanding"

Before moving on, you should be able to sit down and write this without looking at notes:

```
CREATE TABLE documents (  
  id TEXT PRIMARY KEY,  
  name TEXT NOT NULL,  
  document_type TEXT NOT NULL,  
  version TEXT,  
  status TEXT  
);
```

Then write:

```
INSERT INTO documents (...);
```

```
SELECT *  
FROM documents;
```

```
SELECT *  
FROM documents  
WHERE id = ...;
```

```
UPDATE documents  
SET version = ...  
WHERE id = ...;
```

```
DELETE FROM documents  
WHERE id = ...;
```

Then explain:

Primary Key
Foreign Key
NOT NULL
UNIQUE
CHECK
Transaction

COMMIT
ROLLBACK
Index
JOIN
WHERE
GROUP BY
HAVING

without memorizing definitions word-for-word.

Most importantly:

You should be able to explain this architecture:

```
FastAPI
  ↓
DocumentService
  ↓
DocumentRepository
  ↓
SQL
  ↓
PostgreSQL
```

and explain why each layer exists.

That is the SQL/backend foundation we need before you start treating PostgreSQL as part of a production system. The Day 5 plan specifically calls for this transition while keeping the existing API/service architecture stable.